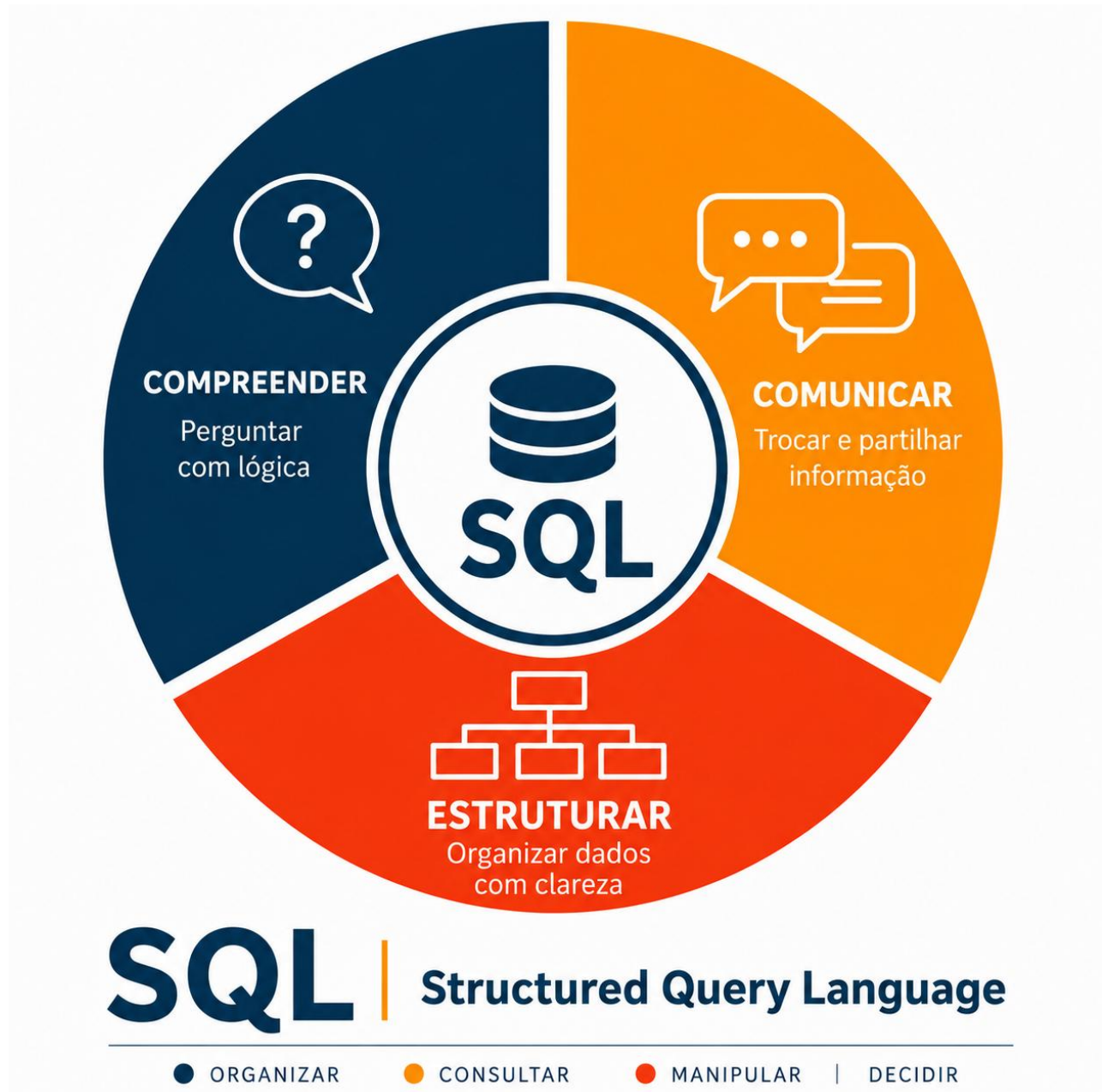


UFCD 10788

Fundamentos da linguagem SQL



COMPREENDER

COMUNICAR

ESTRUTURAR

SQL | Structured Query Language

Organizar | Consultar | Manipular | Decidir

Índice

Introdução	1
Como usar este manual	1
Percurso do manual	1
1. Introdução a bases de dados	2
1.1 O que são dados e informação	2
1.2 Exemplos de bases de dados no dia a dia	2
1.3 Porque se usam bases de dados.....	2
1.4 Como a informação pode ser organizada.....	3
1.5 Primeira ligação ao SQL	3
1.6 Conceitos essenciais deste ponto	3
1.7 Atividade pratica	4
1.8 Verificação da aprendizagem.....	4
Síntese do 1	4
2. Ambientes de bases de dados.....	5
2.1 O que faz parte de um ambiente de base de dados	5
2.2 SGBD: sistema de gestão de bases de dados	5
2.3 Ambientes locais, em rede e na nuvem	5
2.4 Cliente, servidor e base de dados.....	6
2.5 Ferramentas para trabalhar com SQL	6
2.6 Conceitos essenciais deste tema	6
2.7 Atividade pratica	7
2.8 Verificação da aprendizagem.....	7
Síntese do 2.....	7
3. Terminologia de bases de dados relacionais	8
3.1 Tabela, campo e registo	8
3.2 Chave primaria	8
3.3 Chave estrangeira	8
3.4 Relações entre tabelas	9
3.5 Integridade dos dados.....	9
3.6 Conceitos essenciais deste tema	9
3.7 Atividade pratica	9
3.8 Verificação da aprendizagem.....	10

Síntese do 3.....	10
4. Planeamento e desenho de bases de dados.....	11
4.1 Levantamento de necessidades	11
4.2 Identificação de entidades	11
4.3 Definição de campos.....	11
4.4 Desenho das relações	12
4.5 Do desenho ao SQL	12
4.6 Conceitos essenciais deste tema	12
4.7 Atividade pratica	13
4.8 Verificação da aprendizagem.....	13
Síntese do 4.....	13
5. Introdução ao SQL.....	14
5.1 Para que serve o SQL.....	14
5.2 Tipos principais de comandos SQL.....	14
5.3 Primeira consulta com SELECT.....	14
5.4 Ler uma instrução SQL.....	15
5.5 SELECT, FROM, WHERE e ORDER BY	15
5.6 Exemplo completo de consulta	15
5.7 Conceitos essenciais deste tema	16
5.8 Atividade pratica	16
5.9 Verificação da aprendizagem.....	16
Síntese do 5.....	16
6. Criação de bases de dados.....	17
6.1 Antes de criar uma base de dados.....	17
6.2 Nome da base de dados.....	17
6.3 Comando CREATE DATABASE.....	17
6.4 Selecionar a base de dados	18
6.5 Planeamento inicial da base de dados.....	18
6.6 Exemplo de script inicial.....	18
6.7 Conceitos essenciais deste tema	19
6.8 Atividade pratica	19
6.9 Verificação da aprendizagem.....	19
Síntese do 6.....	19
7. Tabelas e integridade de dados	20

7.1 O que é uma tabela.....	20
7.2 Criar uma tabela com CREATE TABLE	20
7.3 Tipos de dados.....	20
7.4 Chave primaria	21
7.5 Regras de integridade.....	21
7.6 Chave estrangeira e relação entre tabelas	21
7.7 Conceitos essenciais deste tema	22
7.8 Atividade pratica	22
7.9 Verificação da aprendizagem.....	22
Síntese do 7	23
8. Fundamentos de Transact-SQL	24
8.1 O que distingue Transact-SQL.....	24
8.2 Instruções, scripts e batches	24
8.3 Comentários em T-SQL.....	24
8.4 Inserir dados com INSERT	25
8.5 Alterar dados com UPDATE	25
8.6 Apagar dados com DELETE	25
8.7 Comandos essenciais	26
8.8 Conceitos essenciais deste tema	26
8.9 Atividade pratica	26
8.10 Verificação da aprendizagem	26
Síntese do 8.....	27
9. Filtragem e ordenação de dados.....	28
9.1 Filtrar dados com WHERE	28
9.2 Operadores de comparação	28
9.3 Combinar condições com AND e OR.....	28
9.4 Pesquisar texto com LIKE.....	29
9.5 IN e BETWEEN	29
9.6 Ordenar dados com ORDER BY	30
9.7 Resumo dos filtros e ordenação	30
9.8 Conceitos essenciais deste tema	30
9.9 Atividade pratica	30
9.10 Verificação da aprendizagem	31
Síntese do 9.....	31

Conclusão	32
1. O que foi trabalhado.....	32
2. Competências desenvolvidas.....	32
3. Cuidados importantes ao trabalhar com SQL.....	32
4. Exemplo de percurso pratico final	33
5. Autoavaliação.....	33
6. Continuidade da aprendizagem	33
Mensagem final.....	34
Bibliografia	34

Introdução

A UFCD 10788 - Fundamentos da linguagem SQL introduz os conceitos essenciais para compreender como a informação pode ser organizada, guardada e consultada numa base de dados relacional.

Este manual foi construído para apoiar uma aprendizagem gradual. Começa pela noção de base de dados, passa pela terminologia relacional, pelo planeamento da estrutura e pela criação de bases de dados e tabelas, terminando com comandos SQL para consultar, filtrar, ordenar e manipular dados.

Objetivo do manual: Ajudar o formando a compreender a lógica das bases de dados e a escrever comandos SQL simples com clareza e segurança.

Como usar este manual

Cada parte do manual trabalha um conteúdo específico da UFCD. A leitura deve ser acompanhada por prática, porque SQL aprende-se melhor quando se escreve, testa, corrige e explica cada comando.

1. Ler primeiro a explicação de cada conteúdo.
2. Observar os exemplos SQL apresentados.
3. Resolver a atividade prática de cada parte.
4. Responder às perguntas de verificação.
5. Voltar aos exemplos sempre que surgir uma dúvida.

Percurso do manual

- 1 - Introdução a bases de dados.
- 2 - Ambientes de bases de dados.
- 3 - Terminologia de bases de dados relacionais.
- 4 - Planeamento e desenho de bases de dados.
- 5 - Introdução ao SQL.
- 6 - Criação de bases de dados.
- 7 - Tabelas e integridade de dados.
- 8 - Fundamentos de Transact-SQL.
- 9 - Filtragem e ordenação de dados.

1. Introdução a bases de dados

Uma base de dados é um conjunto organizado de dados. Em vez de guardar informação de forma solta, numa lista confusa ou em muitos ficheiros separados, a base de dados permite guardar, procurar, alterar e analisar informação de forma mais segura e mais rápida.

Nesta UFCD, a base de dados vai ser estudada como ponto de partida para a linguagem SQL. Antes de escrever comandos, é importante perceber que o SQL serve para comunicar com bases de dados: consultar dados, criar estruturas, inserir registos, alterar informação e apagar dados quando isso faz sentido.

Ideia-chave: Uma base de dados não é apenas um sítio onde se guardam dados. É uma forma organizada de representar uma realidade.

1.1 O que são dados e informação

Dados são valores isolados. Podem ser nomes, números, datas, códigos, moradas ou resultados. Só por si, podem ter pouco significado.

Informação aparece quando esses dados são organizados e interpretados. Por exemplo, o número 22 é apenas um dado. Se soubermos que corresponde à idade de uma formanda chamada Ana Santos, esse dado passa a ter contexto.

- Dado: valor registado, como 22, Lisboa ou Ana Santos.
- Informação: dado com contexto, por exemplo: Ana Santos tem 22 anos e vive em Lisboa.
- Base de dados: estrutura organizada onde esses dados ficam guardados para poderem ser usados.

1.2 Exemplos de bases de dados no dia a dia

Mesmo sem repararmos, usamos bases de dados em muitas situações. Sempre que uma aplicação precisa de guardar informação e depois encontrá-la rapidamente, existe normalmente uma base de dados por trás.

- Uma escola guarda dados de alunos, turmas, formadores, horários e notas.
- Uma loja online guarda produtos, clientes, encomendas e pagamentos.
- Um centro de saúde guarda utentes, consultas, médicos e histórico de atendimento.
- Uma biblioteca guarda livros, autores, leitores e empréstimos.
- Uma plataforma de formação guarda utilizadores, cursos, tarefas, presentes e resultados.

1.3 Porque se usam bases de dados

As bases de dados ajudam a resolver problemas comuns quando há muita informação para gerir. Sem uma estrutura organizada, é fácil repetir dados, perder informação, escrever valores errados ou demorar muito tempo a encontrar aquilo que se procura.

- Organizar dados de forma clara.
- Evitar repetições desnecessárias.
- Procurar informação rapidamente.
- Atualizar dados sem ter de alterar muitos ficheiros.
- Proteger melhor a informação.
- Permitir que várias pessoas ou aplicações usem os mesmos dados.

Exemplo simples: Se uma turma tiver 25 formandos, uma folha simples pode ser suficiente. Se a entidade tiver centenas de formandos, várias turmas, presenças, notas e certificados, uma base de dados torna-se muito mais adequada.

1.4 Como a informação pode ser organizada

Numa base de dados relacional, a informação costuma ser organizada em tabelas. Uma tabela parece-se com uma folha com linhas e colunas, mas segue regras que ajudam a manter os dados consistentes.

No exemplo seguinte, cada linha representa um formando e cada coluna representa uma característica desse formando.

id_formando	nome	idade	localidade
1	Ana Santos	22	Lisboa
2	Bruno Costa	19	Setúbal
3	Carla Pereira	31	Porto
4	Diogo Martins	17	Braga

Neste exemplo, a coluna id_formando identifica cada formando. O nome, a idade e a localidade são dados associados a esse formando. Mais tarde, com SQL, poderíamos consultar apenas os formandos de uma determinada localidade ou ordenar a lista por nome.

1.5 Primeira ligação ao SQL

SQL significa Structured Query Language, ou linguagem estruturada de consulta. É uma linguagem usada para trabalhar com bases de dados relacionais. Para já, o mais importante é perceber que uma instrução SQL é uma ordem dada a base de dados.

Por exemplo, se quisermos ver o nome e a localidade dos formandos com idade igual ou superior a 18 anos, poderíamos escrever uma consulta como esta:

```
SELECT nome, localidade
FROM Formandos
WHERE idade >= 18
ORDER BY nome;
```

Neste momento não é necessário decorar o comando. O objetivo é perceber a ideia: a base de dados tem dados organizados e o SQL permite pedir apenas a informação de que precisamos.

1.6 Conceitos essenciais deste ponto

Conceito	Explicação simples
Dados	Valores registados, como nomes, números, datas ou códigos.
Informação	Dados organizados e interpretados com significado.
Base de dados	Conjunto organizado de dados que pode ser consultado e atualizado.

Tabela	Estrutura com colunas e linhas onde se guardam dados relacionados.
---------------	--

1.7 Atividade pratica

A atividade seguinte ajuda a aplicar a ideia principal deste ponto: identificar dados e pensar numa forma simples de os organizar.

6. Escolhe um contexto simples, como uma turma, uma biblioteca, uma loja ou uma agenda de contactos.
7. Indica cinco dados que fariam sentido guardar nesse contexto.
8. Organiza esses dados numa tabela simples, com nomes de colunas.
9. Assinala qual seria o campo mais adequado para identificar cada registo.
10. Explica, por palavras tuas, que pergunta poderias fazer a essa base de dados.

Resultado esperado: No final, o formando deve conseguir explicar o que é uma base de dados e dar um exemplo simples de dados organizados numa tabela.

1.8 Verificação da aprendizagem

Responde por palavras tuas:

11. O que é uma base de dados?
12. Qual é a diferença entre dados e informação?
13. Dá dois exemplos de bases de dados que uses no dia a dia.
14. Porque é importante organizar os dados antes de os consultar?
15. Para que pode servir a linguagem SQL?

Síntese do 1

Uma base de dados permite guardar dados de forma organizada para que possam ser consultados, alterados e usados com mais facilidade. Ao longo da UFCD, estes conceitos vão servir de base para compreender tabelas, relações e comandos SQL.

Referencia de enquadramento: Catálogo Nacional de Qualificações, UFCD 10788 - Fundamentos da linguagem SQL.

2. Ambientes de bases de dados

Um ambiente de base de dados é o conjunto de componentes que permite guardar, consultar, alterar e proteger dados. Quando uma pessoa usa uma aplicação, muitas vezes não contacta diretamente com a base de dados. A aplicação envia pedidos, o sistema de gestão de bases de dados interpreta esses pedidos e devolve os resultados.

Nesta parte da UFCD, o objetivo é perceber onde vive a base de dados, que ferramentas podem ser usadas para aceder aos dados e porque é que nem todos os ambientes funcionam da mesma forma.

Ideia-chave: A base de dados é apenas uma parte do ambiente. Para trabalhar com SQL, também precisamos de uma ferramenta, de permissões e de um sistema que interprete os comandos.

2.1 O que faz parte de um ambiente de base de dados

Um ambiente de base de dados pode ser simples ou complexo. Num exercício de formação, podemos ter apenas uma ferramenta local no computador. Numa empresa, a base de dados pode estar num servidor, ser acedida por várias aplicações e ter regras de segurança mais rigorosas.

Os principais elementos são a aplicação, o cliente SQL, o servidor de base de dados, a própria base de dados e os utilizadores com diferentes permissões.

- Aplicação: interface usada pelo utilizador, como um site ou programa de gestão.
- Cliente SQL: ferramenta onde se escrevem e executam comandos SQL.
- Servidor: computador ou serviço que guarda e processa a base de dados.
- SGBD: software que gere a base de dados, as tabelas, os acessos e as operações.
- Utilizador: pessoa ou sistema com permissões para consultar ou alterar dados.

2.2 SGBD: sistema de gestão de bases de dados

SGBD significa Sistema de Gestão de Bases de Dados. É o software que permite criar bases de dados, guardar informação, executar consultas, controlar acessos e manter os dados organizados.

- Microsoft SQL Server: muito usado em ambientes empresariais e em formação com Transact-SQL.
- MySQL ou MariaDB: frequentes em sites e aplicações web.
- PostgreSQL: usado em projetos que precisam de robustez e flexibilidade.
- SQLite: base de dados leve, muitas vezes guardada num único ficheiro.
- Microsoft Access: usado em contextos mais simples e com interface integrada.

Nota: Nesta UFCD, quando se fala em Transact-SQL, a referência principal é o SQL usado no ecossistema Microsoft SQL Server.

2.3 Ambientes locais, em rede e na nuvem

A base de dados pode estar no próprio computador, num servidor da rede ou num serviço na nuvem. A escolha depende do objetivo, do número de utilizadores, da segurança necessária e dos recursos disponíveis.

- Ambiente local: a base de dados funciona no computador do utilizador.
- Ambiente em rede: a base de dados está num servidor acessível por vários computadores.
- Ambiente na nuvem: a base de dados está alojada num serviço online.

- Ambiente de desenvolvimento: usado para testes e aprendizagem.
- Ambiente de produção: usado por utilizadores reais e deve ser mais protegido.

Alerta: Não se deve testar comandos perigosos diretamente numa base de dados de produção. Primeiro deve usar-se uma base de dados de treino ou desenvolvimento.

2.4 Cliente, servidor e base de dados

Muitos ambientes de bases de dados seguem uma lógica cliente-servidor. O cliente é a ferramenta usada para enviar pedidos. O servidor recebe esses pedidos, executa-os e devolve uma resposta.

Por exemplo, quando escrevemos uma consulta SQL num cliente, essa consulta é enviada para o servidor. O servidor verifica se o utilizador tem permissão, executa a consulta e devolve os dados encontrados.

Elemento	Função	Exemplo
Aplicação	Permite ao utilizador introduzir ou consultar dados.	site, app, programa de gestão
Cliente SQL	Ferramenta usada para escrever comandos SQL.	SQL Server Management Studio, Azure Data Studio
Servidor de base de dados	Guarda e processa os dados.	SQL Server, MySQL, PostgreSQL
Base de dados	Conjunto organizado de tabelas e outros objetos.	base de dados de uma escola

Esta separação ajuda a perceber porque é que uma base de dados pode ser usada por várias aplicações ao mesmo tempo, desde que existam regras de acesso, controlo e segurança.

2.5 Ferramentas para trabalhar com SQL

Para escrever SQL, normalmente usamos uma ferramenta própria. Essa ferramenta permite ligar ao servidor, escolher a base de dados, escrever comandos, executar consultas e observar os resultados.

Uma consulta simples executada num cliente SQL poderia ser:

```
-- Ver todas as bases de dados disponíveis no servidor
SELECT name
FROM sys.databases;
```

Este exemplo mostra que o SQL não serve apenas para consultar tabelas com dados de negócio. Também pode ser usado para conhecer o próprio ambiente onde estamos a trabalhar.

2.6 Conceitos essenciais deste tema

Conceito	Explicação simples
SGBD	Software que gere bases de dados e permite executar comandos SQL.
Cliente SQL	Ferramenta usada para ligar ao servidor e escrever consultas.

Servidor	Sistema que guarda, processa e protege a base de dados.
Produção	Ambiente real, usado por pessoas ou aplicações no dia a dia.

2.7 Atividade pratica

A atividade seguinte ajuda a distinguir os elementos de um ambiente de base de dados e a perceber o caminho que uma consulta percorre.

16. Escolhe uma aplicação que uses no dia a dia, como Moodle, loja online ou agenda digital.
17. Identifica que dados essa aplicação provavelmente guarda.
18. Indica quem seria o utilizador, qual seria a aplicação e onde poderia estar a base de dados.
19. Explica se esse ambiente seria local, em rede ou na nuvem.
20. Indica dois cuidados de segurança que fariam sentido nesse ambiente.

Resultado esperado: No final, o formando deve conseguir explicar a diferença entre aplicação, cliente SQL, servidor, SGBD e base de dados.

2.8 Verificação da aprendizagem

Responde por palavras tuas:

21. O que é um SGBD?
22. Qual é a diferença entre cliente SQL e servidor de base de dados?
23. Dá um exemplo de ambiente local e um exemplo de ambiente na nuvem.
24. Porque é perigoso testar comandos numa base de dados de produção?
25. Que ferramenta poderias usar para escrever comandos SQL?

Síntese do 2

Um ambiente de base de dados junta vários componentes: aplicações, clientes SQL, servidores, SGBD, utilizadores e regras de acesso. Compreender este ambiente ajuda a trabalhar com SQL de forma mais segura, sabendo onde os dados estão, quem pode aceder e que cuidados devem ser respeitados.

Referencia de enquadramento: Catálogo Nacional de Qualificações, UFCD 10788 - Fundamentos da linguagem SQL.

3. Terminologia de bases de dados relacionais

Para trabalhar com bases de dados relacionais e com SQL, é essencial conhecer a terminologia mais usada. Estes conceitos funcionam como uma linguagem comum: ajudam a perceber como os dados estão organizados e como podem ser consultados ou alterados.

Uma base de dados relacional organiza a informação em tabelas. Cada tabela guarda dados sobre um tema, como formandos, turmas, cursos, produtos ou encomendas. As tabelas podem relacionar-se entre si através de campos comuns.

Ideia-chave: Numa base de dados relacional, os dados ficam organizados em tabelas e as relações ajudam a evitar repetições e incoerências.

3.1 Tabela, campo e registo

Uma tabela é uma estrutura onde se guardam dados sobre um determinado assunto. Pode ser comparada a uma grelha com colunas e linhas, mas numa base de dados cada coluna é definida com um nome e, normalmente, com um tipo de dados.

Um campo corresponde a uma coluna da tabela. Um registo corresponde a uma linha da tabela. Por exemplo, numa tabela de formandos, os campos podem ser `id_formando`, `nome`, `data_nascimento` e `localidade`. Cada formando será um registo.

- Tabela: conjunto organizado de dados sobre um tema.
- Campo: coluna da tabela, como `nome`, `idade` ou `localidade`.
- Registo: linha da tabela, correspondente a uma entidade concreta.
- Valor: dado guardado numa célula, como `Lisboa` ou `22`.

3.2 Chave primaria

A chave primaria é o campo, ou conjunto de campos, que identifica cada registo de forma única. Isto significa que não devem existir dois registos com a mesma chave primaria na mesma tabela.

- Ajuda a distinguir registos semelhantes.
- Evita duplicação de identificadores.
- Permite relacionar dados com outras tabelas.
- Normalmente usa nomes como `id_formando`, `id_turma` ou `id_produto`.

Exemplo: Numa tabela `Formandos`, o campo `id_formando` pode ser a chave primaria, porque identifica cada formando de forma única.

3.3 Chave estrangeira

A chave estrangeira é um campo que cria uma ligação entre duas tabelas. Ela aponta para a chave primaria de outra tabela. Desta forma, conseguimos relacionar informação sem repetir todos os dados em várias tabelas.

- A tabela `Turmas` pode ter `id_turma` como chave primaria.
- A tabela `Formandos` pode ter `id_turma` como chave estrangeira.
- Assim, cada formando pode ficar associado a uma turma.
- A relação evita escrever o nome completo da turma em todos os registos.

Alerta: Uma chave estrangeira deve apontar para um registo existente. Caso contrário, a base de dados pode ficar incoerente.

3.4 Relações entre tabelas

As relações indicam como os dados de uma tabela se ligam aos dados de outra tabela. Esta ideia é uma das bases mais importantes das bases de dados relacionais.

No exemplo seguinte, a tabela Formandos inclui o campo `id_turma`. Esse campo permite associar cada formando a uma turma registada noutra tabela.

<code>id_formando</code>	<code>nome</code>	<code>id_turma</code>	<code>localidade</code>
1	Ana Santos	T01	Lisboa
2	Bruno Costa	T01	Setúbal
3	Carla Pereira	T02	Porto

Com esta estrutura, se o nome da turma mudar, não é necessário alterar todos os formandos. Basta alterar a tabela Turmas, mantendo a relação através do identificador.

3.5 Integridade dos dados

A integridade dos dados significa que a informação deve ser correta, coerente e válida. As regras de integridade ajudam a evitar erros, duplicações e relações impossíveis.

Em SQL, uma tabela simples com chave primaria pode ser criada assim:

```
CREATE TABLE Turmas (  
    id_turma INT PRIMARY KEY,  
    designacao VARCHAR(50) NOT NULL  
);
```

Neste exemplo, `id_turma` identifica cada turma de forma única. A indicação `NOT NULL` significa que a designação da turma não deve ficar vazia.

3.6 Conceitos essenciais deste tema

Conceito	Explicação simples
Tabela	Estrutura onde se guardam dados sobre um tema.
Campo	Coluna de uma tabela.
Registo	Linha de uma tabela.
Chave primaria	Campo que identifica cada registo de forma única.

3.7 Atividade pratica

A atividade seguinte ajuda a aplicar a terminologia das bases de dados relacionais a uma situação concreta.

26. Imagina uma base de dados para gerir uma biblioteca.

27. Indica duas tabelas que fariam sentido existir.
28. Para cada tabela, escreve pelo menos quatro campos.
29. Escolhe uma chave primaria para cada tabela.
30. Indica uma possível chave estrangeira para relacionar as duas tabelas.

Resultado esperado: No final, o formando deve conseguir identificar tabelas, campos, registos, chaves primarias, chaves estrangeiras e relações simples.

3.8 Verificação da aprendizagem

Responde por palavras tuas:

31. O que é uma tabela?
32. Qual é a diferença entre campo e registo?
33. Para que serve uma chave primaria?
34. O que é uma chave estrangeira?
35. Porque é importante manter a integridade dos dados?

Síntese do 3

A terminologia das bases de dados relacionais permite compreender como a informação é organizada. Tabelas guardam dados, campos representam características, registos representam linhas concretas, e as chaves permitem identificar e relacionar informação com rigor.

Referencia de enquadramento: Catálogo Nacional de Qualificações, UFCD 10788 - Fundamentos da linguagem SQL.

4. Planeamento e desenho de bases de dados

Antes de criar uma base de dados, é importante planejar. O planeamento ajuda a perceber que informação deve ser guardada, como essa informação deve ser organizada e que relações existem entre os diferentes dados.

Um bom desenho evita problemas frequentes, como dados repetidos, campos mal escolhidos, tabelas demasiado grandes ou dificuldade em obter respostas através de consultas SQL.

Ideia-chave: Planear primeiro e criar depois. Uma base de dados bem pensada é mais fácil de consultar, manter e corrigir.

4.1 Levantamento de necessidades

O primeiro passo é compreender o problema. Antes de pensar em tabelas, devemos perceber que informação é necessária, quem vai usar a base de dados e que perguntas devem poder ser respondidas.

Por exemplo, se estivermos a criar uma base de dados para uma formação, podemos precisar de guardar formandos, turmas, UFCD, inscrições, presenças e resultados.

- Que dados precisam de ser guardados?
- Quem vai inserir, consultar ou alterar os dados?
- Que relatórios ou consultas serão necessários?
- Que dados não devem ser repetidos?
- Que regras de validação devem existir?

4.2 Identificação de entidades

Uma entidade representa algo sobre o qual queremos guardar informação. Em muitos casos, cada entidade dará origem a uma tabela. Exemplos de entidades são Formando, Turma, UFCD, Produto, Cliente ou Encomenda.

- Formando: pessoa que frequenta a formação.
- Turma: grupo onde os formandos estão inscritos.
- UFCD: unidade de formação com código, designação e carga horaria.
- Inscrição: ligação entre um formando e uma turma.
- Presença: registo de assiduidade numa sessão.

Exemplo: Se uma base de dados precisa de guardar formandos e turmas, provavelmente deve existir uma tabela Formandos e uma tabela Turmas.

4.3 Definição de campos

Depois de identificar as entidades, devemos escolher os campos de cada tabela. Cada campo deve guardar apenas uma informação clara e específica. Campos demasiado vagos tornam a base de dados mais difícil de usar.

- Usar nomes claros, como nome, email, data_nascimento ou id_turma.
- Evitar guardar várias informações no mesmo campo.
- Escolher o tipo de dados adequado: texto, número, data, valor lógico.
- Indicar que campos são obrigatórios.

- Definir que campo identifica cada registo.

Alerta: Um campo chamado dados formando e pouco claro. E melhor separar em campos como nome, email, telefone e localidade.

4.4 Desenho das relações

As relações mostram como as tabelas se ligam entre si. Este passo é muito importante porque evita duplicar informação e permite consultar dados combinados de várias tabelas.

Num caso de formação, um formando pode estar inscrito numa turma, uma turma pode trabalhar uma UFCD e uma inscrição pode ligar um formando a uma turma. Esse desenho deve ser pensado antes da criação das tabelas.

Entidade	Campos possíveis	Observação
Formando	id_formando, nome, email	Cada formando deve ter identificador único.
Turma	id_turma, designação, data_inicio	Uma turma pode ter vários formandos.
UFCD	id_ufcd, código, designação	A UFCD pode estar associada a uma turma.
Inscrição	id_inscricao, id_formando, id_turma	Liga formandos a turmas.

A tabela Inscrição aparece porque a relação entre formandos e turmas pode precisar de dados próprios, como data de inscrição, estado ou resultado.

4.5 Do desenho ao SQL

Depois do planeamento, o desenho pode ser transformado em SQL. A criação das tabelas deve respeitar as entidades, campos, chaves e relações que foram definidos.

Um primeiro rascunho SQL poderia começar assim:

```
CREATE TABLE Formandos (  
  id_formando INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(120)  
);
```

Este comando ainda é simples, mas já mostra que o planeamento influencia diretamente a estrutura criada em SQL.

4.6 Conceitos essenciais deste tema

Conceito	Explicação simples
Entidade	Algo sobre o qual queremos guardar informação.
Campo	Informação específica guardada numa tabela.
Relação	Ligação entre tabelas através de campos comuns.

Regra de negócio	Condição que a base de dados deve respeitar.
-------------------------	--

4.7 Atividade pratica

A atividade seguinte ajuda a transformar uma necessidade simples num primeiro desenho de base de dados.

36. Escolhe um contexto: biblioteca, loja, escola, clínica ou formação.
37. Escreve que informação precisa de ser guardada.
38. Identifica pelo menos três entidades.
39. Define quatro campos para cada entidade.
40. Indica as chaves primarias e uma relação entre duas tabelas.

Resultado esperado: No final, o formando deve conseguir desenhar uma estrutura inicial de base de dados antes de escrever comandos SQL.

4.8 Verificação da aprendizagem

Responde por palavras tuas:

41. Porque é importante planear uma base de dados antes de a criar?
42. O que é uma entidade?
43. Que cuidados devemos ter ao definir campos?
44. Para que servem as relações entre tabelas?
45. Como o desenho da base de dados influencia os comandos SQL?

Síntese do 4

O planeamento e o desenho de uma base de dados ajudam a organizar a informação antes da criação das tabelas. Este trabalho permite definir entidades, campos, chaves e relações de forma mais clara, reduzindo erros e facilitando as consultas SQL futuras.

Referencia de enquadramento: Catálogo Nacional de Qualificações, UFCD 10788 - Fundamentos da linguagem SQL.

5. Introdução ao SQL

SQL significa Structured Query Language, ou Linguagem Estruturada de Consulta. É uma linguagem usada para comunicar com bases de dados relacionais. Através de SQL podemos consultar, criar, inserir, alterar e apagar dados.

Nesta introdução, o foco principal está na leitura de comandos simples. O objetivo não é decorar tudo de imediato, mas compreender a lógica: um comando SQL é uma instrução clara à base de dados.

Ideia-chave: SQL permite fazer perguntas à base de dados. A resposta aparece sob a forma de resultados organizados em linhas e colunas.

5.1 Para que serve o SQL

SQL serve para trabalhar com os dados guardados numa base de dados relacional. Com SQL podemos pedir informação específica, criar estruturas e manipular registos.

Por exemplo, numa base de dados de formação, podemos pedir a lista de formandos, procurar apenas os formandos de uma turma, ordenar nomes ou verificar que UFCD estão associadas a determinado percurso.

- Consultar dados com SELECT.
- Criar bases de dados e tabelas.
- Inserir novos registos.
- Alterar dados existentes.
- Apagar dados quando necessário e com cuidado.

5.2 Tipos principais de comandos SQL

Os comandos SQL podem ser agrupados por finalidade. Esta divisão ajuda a perceber se estamos apenas a consultar dados, a alterar conteúdo ou a modificar a própria estrutura da base de dados.

- DQL: comandos de consulta, como SELECT.
- DDL: comandos de definição de estrutura, como CREATE TABLE.
- DML: comandos de manipulação de dados, como INSERT, UPDATE e DELETE.
- DCL: comandos ligados a permissões e acessos.
- TCL: comandos ligados a transações, como COMMIT e ROLLBACK.

Nota: No início da aprendizagem, o comando SELECT costuma ser o mais importante, porque permite consultar dados sem os alterar.

5.3 Primeira consulta com SELECT

O comando SELECT é usado para consultar dados. A sua forma mais simples indica que colunas queremos ver e em que tabela esses dados se encontram.

Uma primeira consulta pode ter esta estrutura:

```
SELECT nome, localidade  
FROM Formandos;
```

Neste exemplo, a base de dados devolve os valores das colunas nome e localidade que existem na tabela Formandos.

- SELECT indica as colunas a apresentar.
- FROM indica a tabela de origem.
- O ponto e vírgula, pode ser usado para terminar a instrução.
- A consulta não altera os dados; apenas apresenta resultados.

5.4 Ler uma instrução SQL

Uma instrução SQL deve ser lida por partes. Isto ajuda a compreender o que a base de dados está a receber como pedido.

- Identificar o comando principal.
- Perceber que colunas estão a ser pedidas.
- Identificar a tabela usada.
- Verificar se existe filtro.
- Verificar se existe ordenação.

Alerta: Antes de executar uma instrução SQL, devemos confirmar se ela apenas consulta dados ou se também altera/apaga informação.

5.5 SELECT, FROM, WHERE e ORDER BY

Embora este ponto seja introdutório, é útil conhecer desde já algumas partes frequentes de uma consulta. O SELECT e o FROM são a base; o WHERE filtra resultados; o ORDER BY organiza a apresentação.

A tabela seguinte resume a função de cada parte:

Parte	Função	Exemplo
SELECT	Indica as colunas que queremos ver.	SELECT nome
FROM	Indica a tabela onde os dados estão.	FROM Formandos
WHERE	Permite filtrar resultados.	WHERE idade >= 18
ORDER BY	Ordena os resultados.	ORDER BY nome

Com estas quatro partes já é possível construir muitas consultas simples e úteis no dia a dia.

5.6 Exemplo completo de consulta

Agora podemos juntar as partes numa consulta um pouco mais completa. O objetivo é pedir apenas os formandos maiores de idade e ordenar os resultados pelo nome.

Exemplo:

```
SELECT nome, idade, localidade
FROM Formandos
WHERE idade >= 18
ORDER BY nome;
```

Lida em linguagem natural, esta consulta significa: mostra o nome, a idade e a localidade dos formandos que tenham 18 anos ou mais, ordenados por nome.

5.7 Conceitos essenciais deste tema

Conceito	Explicação simples
SQL	Linguagem usada para comunicar com bases de dados relacionais.
SELECT	Comando usado para consultar dados.
FROM	Indica a tabela de onde os dados são obtidos.
WHERE	Permite filtrar os resultados de uma consulta.

5.8 Atividade pratica

A atividade seguinte ajuda a ler e construir consultas SQL simples.

46. Observa uma tabela chamada Produtos com os campos id_produto, nome, preço e stock.
47. Escreve uma consulta para apresentar apenas o nome e o preço dos produtos.
48. Escreve uma consulta para apresentar todos os produtos com stock superior a 0.
49. Acrescenta uma ordenação pelo nome do produto.
50. Explica por palavras tuas o que cada consulta faz.

Resultado esperado: No final, o formando deve conseguir ler uma consulta SELECT simples e identificar as partes principais da instrução.

5.9 Verificação da aprendizagem

Responde por palavras tuas:

51. O que significa SQL?
52. Para que serve o comando SELECT?
53. Qual é a função do FROM?
54. Para que serve o WHERE?
55. Porque devemos ter cuidado antes de executar comandos que alteram dados?

Síntese do 5

SQL é a linguagem usada para comunicar com bases de dados relacionais. Neste primeiro contacto, o comando SELECT permite consultar dados, FROM indica a tabela usada, WHERE filtra resultados e ORDER BY organiza a apresentação. Estes elementos formam a base das primeiras consultas.

6. Criação de bases de dados

Criar uma base de dados e preparar um espaço organizado onde as tabelas, os dados e outros objetos vão ficar guardados. Antes de criar tabelas, é necessário existir uma base de dados onde essas estruturas possam ser criadas.

Nesta fase, o foco está na criação da base de dados e na preparação do contexto de trabalho. A criação de tabelas será aprofundada no ponto seguinte.

Ideia-chave: Criar uma base de dados e como preparar uma pasta estruturada para um projeto: antes de guardar informação, é preciso definir onde ela vai ficar.

6.1 Antes de criar uma base de dados

Antes de criar uma base de dados, devemos confirmar o objetivo do projeto. A base de dados deve ter uma finalidade clara: gerir uma formação, uma biblioteca, uma loja, uma clínica ou outro conjunto de informação.

Também é importante perceber se estamos a trabalhar num ambiente de treino, desenvolvimento ou produção. Em formação, o mais adequado é usar uma base de dados de treino.

- Definir o objetivo da base de dados.
- Escolher um nome claro e sem ambiguidades.
- Confirmar se é uma base de dados de treino ou de produção.
- Identificar que tabelas podem existir depois.
- Garantir que temos permissão para criar a base de dados.

6.2 Nome da base de dados

O nome da base de dados deve ser simples, descritivo e fácil de reconhecer. Em muitos ambientes, evita-se usar acentos, espaços e caracteres especiais nos nomes técnicos.

- Usar nomes curtos, mas compreensíveis.
- Evitar espaços: preferir underscore se for necessário separar palavras.
- Evitar acentos em nomes técnicos.
- Incluir o contexto quando ajuda, como BD_Formacao10788.
- Manter uma convenção de nomes ao longo do projeto.

Nota: O nome apresentado ao utilizador pode ser mais amigável, mas o nome técnico da base de dados deve ser simples e consistente.

6.3 Comando CREATE DATABASE

O comando CREATE DATABASE é usado para criar uma nova base de dados. Este comando pertence ao grupo de comandos que definem estruturas.

Exemplo simples:

```
CREATE DATABASE BD_Formacao10788;
```

Este comando cria uma base de dados chamada BD_Formacao10788. Depois de criada, ainda será necessário selecionar essa base de dados para criar tabelas dentro dela.

- CREATE DATABASE indica que queremos criar uma nova base de dados.
- BD_Formacao10788 é o nome escolhido.
- O ponto e vírgula marca o fim da instrução.
- Este comando cria uma estrutura, não cria ainda tabelas nem dados.

6.4 Selecionar a base de dados

Depois de criar uma base de dados, é necessário garantir que os comandos seguintes serão executados no contexto correto. Em Transact-SQL, é comum usar o comando USE para selecionar a base de dados.

```
USE BD_Formacao10788;
```

A partir desse momento, os comandos seguintes serão executados nessa base de dados, salvo indicação em contrário.

- Confirmar que a base de dados existe.
- Selecionar a base de dados correta.
- Evitar executar comandos no contexto errado.
- Verificar o nome antes de criar tabelas.
- Organizar o script por passos.

Alerta: Executar comandos na base de dados errada pode criar estruturas no sítio errado ou alterar informação que não devia ser alterada.

6.5 Planeamento inicial da base de dados

Criar a base de dados é apenas o início. Antes de avançar para as tabelas, convém registar algumas decisões sobre o projeto.

A tabela seguinte resume decisões simples que ajudam a preparar o trabalho:

Decisão	Porque importa	Exemplo
Nome da base de dados	Ajuda a identificar o projeto.	BD_Formacao10788
Finalidade	Clarifica que dados serão guardados.	Gerir formandos e turmas
Ambiente	Distingue testes de uso real.	Desenvolvimento
Responsável	Indica quem gere e valida a estrutura.	Formador ou equipa técnica

Este registo evita confusões e ajuda a explicar a estrutura a outras pessoas.

6.6 Exemplo de script inicial

Um script inicial pode juntar a criação da base de dados e a seleção do contexto de trabalho. Este tipo de organização ajuda a ler e repetir o processo.

Exemplo:

```
CREATE DATABASE BD_Formacao10788;
GO
```

```
USE BD_Formacao10788;  
GO
```

No SQL Server, GO é usado por algumas ferramentas para separar blocos de comandos. Não é exatamente um comando SQL da base de dados; é uma instrução interpretada pela ferramenta.

6.7 Conceitos essenciais deste tema

Conceito	Explicação simples
CREATE DATABASE	Comando usado para criar uma nova base de dados.
USE	Comando usado para selecionar a base de dados de trabalho.
Script	Conjunto organizado de comandos SQL.
Contexto	Base de dados onde os comandos estão a ser executados.

6.8 Atividade pratica

A atividade seguinte ajuda a planear e escrever os primeiros comandos de criação de uma base de dados.

56. Escolhe um contexto simples para uma base de dados.
57. Define um nome técnico adequado, sem espaços nem acentos.
58. Escreve o comando CREATE DATABASE para esse nome.
59. Escreve o comando USE para selecionar a base de dados.
60. Indica duas tabelas que poderiam ser criadas depois.

Resultado esperado: No final, o formando deve conseguir criar uma base de dados simples e selecionar o contexto de trabalho correto.

6.9 Verificação da aprendizagem

Responde por palavras tuas:

61. Para que serve o comando CREATE DATABASE?
62. Porque é importante escolher bem o nome da base de dados?
63. Para que serve o comando USE?
64. O que pode acontecer se trabalharmos no contexto errado?
65. Que diferença existe entre criar uma base de dados e criar uma tabela?

Síntese do 6

A criação de uma base de dados prepara o espaço onde as tabelas e os dados serão organizados. O comando CREATE DATABASE cria a base de dados e o comando USE seleciona o contexto de trabalho. Antes de avançar, é importante confirmar nomes, objetivos e ambiente.

Referencia de enquadramento: Catálogo Nacional de Qualificações, UFCD 10788 - Fundamentos da linguagem SQL.

7. Tabelas e integridade de dados

Depois de criar uma base de dados, o passo seguinte é criar tabelas. As tabelas são as estruturas onde os dados ficam guardados. Cada tabela deve ter campos bem definidos, tipos de dados adequados e regras que ajudem a manter a informação correta.

A integridade dos dados é o conjunto de regras que ajuda a evitar erros, duplicações, valores em falta e relações impossíveis entre tabelas.

Ideia-chave: Uma tabela bem desenhada protege a qualidade dos dados e torna as consultas SQL mais fiáveis.

7.1 O que é uma tabela

Uma tabela é uma estrutura formada por colunas e linhas. As colunas representam os campos e as linhas representam os registos. Cada tabela deve guardar dados sobre um tema específico.

Por exemplo, numa base de dados de formação, podemos ter uma tabela Formandos, uma tabela Turmas e uma tabela UFCD. Separar os dados por tema facilita a organização e evita repetições.

- Cada tabela deve ter um nome claro.
- Cada campo deve guardar apenas uma informação.
- Cada registo deve representar uma ocorrência concreta.
- A tabela deve ter uma chave primária sempre que possível.
- Os tipos de dados devem ser escolhidos de acordo com a informação guardada.

7.2 Criar uma tabela com CREATE TABLE

O comando CREATE TABLE é usado para criar uma tabela. Ao criar a tabela, indicamos o nome dos campos e o tipo de dados de cada campo.

Exemplo simples:

```
CREATE TABLE Formandos (  
  id_formando INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(120),  
  data_nascimento DATE  
);
```

Neste exemplo, a tabela Formandos tem quatro campos. O campo id_formando identifica cada registo, o nome é obrigatório e os restantes campos podem guardar contacto e data de nascimento.

Nota: Os nomes técnicos das tabelas e campos devem ser claros, consistentes e sem acentos.

7.3 Tipos de dados

O tipo de dados indica que tipo de valor pode ser guardado em cada campo. Escolher o tipo correto é importante para evitar erros e melhorar a qualidade da informação.

Tipo de dado	Uso comum	Exemplo
INT	Números inteiros.	idade, id_formando

VARCHAR	Texto de tamanho variável.	nome, email
DATE	Datas.	data_nascimento
BIT	Valor lógico.	ativo: 0 ou 1

Por exemplo, uma idade deve ser guardada como número inteiro, enquanto um nome deve ser guardado como texto. Uma data de nascimento deve usar um tipo apropriado para datas.

- INT para números inteiros.
- VARCHAR para texto.
- DATE para datas.
- BIT para valores verdadeiro/falso.
- DECIMAL para valores numéricos com casas decimais.

7.4 Chave primaria

A chave primaria identifica cada registo de forma única. Numa tabela de formandos, por exemplo, o campo `id_formando` pode ser a chave primaria.

```
| id_formando INT PRIMARY KEY
```

Isto significa que cada formando deve ter um identificador único. A chave primaria ajuda a distinguir registos e permite criar relações com outras tabelas.

- Não deve repetir valores.
- Não deve ficar vazia.
- Deve identificar claramente cada registo.
- Pode ser usada por outras tabelas como referência.
- Facilita consultas e alterações futuras.

Alerta: Criar uma tabela sem chave primaria pode dificultar a identificação correta dos registos.

7.5 Regras de integridade

As regras de integridade ajudam a garantir que os dados fazem sentido. Estas regras podem impedir valores vazios, duplicados ou relações inválidas.

Algumas regras frequentes são:

- PRIMARY KEY: identifica cada registo de forma única.
- NOT NULL: obriga o preenchimento de um campo.
- UNIQUE: impede valores repetidos num campo.
- FOREIGN KEY: liga uma tabela a outra.
- CHECK: valida uma condição, como idade superior a zero.

Estas regras tornam a base de dados mais segura e reduzem a probabilidade de introduzir informação incorreta.

7.6 Chave estrangeira e relação entre tabelas

A chave estrangeira cria uma relação entre tabelas. Ela permite dizer que um campo de uma tabela aponta para a chave primaria de outra tabela.

Exemplo:

```
CREATE TABLE Turmas (  
    id_turma INT PRIMARY KEY,  
    designacao VARCHAR(80) NOT NULL  
);  
  
CREATE TABLE Formandos (  
    id_formando INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    id_turma INT,  
    FOREIGN KEY (id_turma) REFERENCES Turmas(id_turma)  
);
```

Neste exemplo, a tabela Formandos tem o campo id_turma, que aponta para a tabela Turmas. Assim, cada formando pode ficar associado a uma turma.

7.7 Conceitos essenciais deste tema

Conceito	Explicação simples
CREATE TABLE	Comando usado para criar uma tabela.
Tipo de dados	Define que valores podem ser guardados num campo.
PRIMARY KEY	Regra que identifica cada registo de forma única.
FOREIGN KEY	Regra que liga uma tabela a outra.

7.8 Atividade pratica

A atividade seguinte ajuda a criar tabelas simples e aplicar regras de integridade.

66. Cria o desenho de uma tabela Produtos.
67. Define pelo menos quatro campos para a tabela.
68. Escolhe uma chave primaria.
69. Indica o tipo de dados de cada campo.
70. Acrescenta pelo menos uma regra NOT NULL.

Resultado esperado: No final, o formando deve conseguir criar uma tabela simples com campos, tipos de dados e regras de integridade básicas.

7.9 Verificação da aprendizagem

Responde por palavras tuas:

71. Para que serve o comando CREATE TABLE?
72. Porque devemos escolher tipos de dados adequados?
73. O que é uma chave primaria?
74. Para que serve uma chave estrangeira?
75. Que problema ajuda a evitar a regra NOT NULL?

Síntese do 7

As tabelas guardam os dados de uma base de dados relacional. Ao criar tabelas, devemos definir campos, tipos de dados e regras de integridade. Chaves primarias, chaves estrangeiras e campos obrigatórios ajudam a manter a informação correta, coerente e fácil de consultar.

8. Fundamentos de Transact-SQL

Transact-SQL, muitas vezes abreviado como T-SQL, é a extensão da linguagem SQL usada no ecossistema Microsoft SQL Server. Permite consultar dados, criar estruturas, inserir registros, alterar informação e apagar dados.

Neste ponto, o objetivo é conhecer comandos essenciais e perceber os cuidados necessários antes de executar instruções que modificam dados.

Ideia-chave: Comandos de consulta mostram dados; comandos de manipulação alteram dados. Esta diferença deve estar sempre clara.

8.1 O que distingue Transact-SQL

SQL é a linguagem base usada em muitos sistemas de bases de dados. Transact-SQL é uma variante usada pelo SQL Server, com comandos e recursos específicos desse ambiente.

Muitos comandos são comuns a outros sistemas, como SELECT, INSERT, UPDATE e DELETE. Outros elementos, como GO ou algumas funções, são característicos das ferramentas e do ambiente SQL Server.

- Permite consultar dados com SELECT.
- Permite inserir dados com INSERT.
- Permite alterar dados com UPDATE.
- Permite apagar dados com DELETE.
- Permite organizar comandos em scripts e batches.

8.2 Instruções, scripts e batches

Uma instrução SQL é uma ordem enviada a base de dados. Um script é um conjunto de instruções organizadas num ficheiro ou janela de trabalho. Em ferramentas do SQL Server, é comum encontrar a palavra GO para separar blocos de execução.

Exemplo de script simples:

```
USE BD_Formacao10788;
GO

SELECT nome, localidade
FROM Formandos;
GO
```

Neste exemplo, a base de dados é selecionada primeiro. Depois é executada uma consulta simples à tabela Formandos.

Nota: GO não é exatamente uma instrução SQL enviada à base de dados. É usado por ferramentas do SQL Server para separar blocos.

8.3 Comentários em T-SQL

Comentários ajudam a explicar o que o script faz. São ignorados pela base de dados durante a execução e tornam o código mais fácil de ler.

```
-- Este comentário ocupa uma linha
SELECT nome
```

```
FROM Formandos;  
  
/* Este comentário  
   pode ocupar várias linhas */
```

Usar comentários é uma boa prática, sobretudo em scripts com vários passos.

- Usar `--` para comentários de uma linha.
- Usar `/* ... */` para comentários de várias linhas.
- Comentar partes importantes do script.
- Evitar comentários demasiado longos ou confusos.
- Atualizar comentários quando o código muda.

8.4 Inserir dados com INSERT

O comando INSERT permite adicionar novos registos a uma tabela. Devemos indicar a tabela, os campos e os valores a inserir.

```
INSERT INTO Formandos (id_formando, nome, email)  
VALUES (1, 'Ana Santos', 'ana.santos@email.pt');
```

Neste exemplo, é criado um novo registo na tabela Formandos. Os valores devem respeitar os tipos de dados e as regras da tabela.

- Confirmar que a tabela existe.
- Indicar os campos de forma clara.
- Respeitar a ordem dos valores.
- Usar aspas em valores de texto.
- Garantir que chaves primárias não se repetem.

Alerta: Um INSERT pode falhar se tentar repetir uma chave primária ou deixar vazio um campo obrigatório.

8.5 Alterar dados com UPDATE

O comando UPDATE permite alterar dados existentes. Deve ser usado com muito cuidado, especialmente com a cláusula WHERE, que indica que registos devem ser alterados.

Exemplo:

```
UPDATE Formandos  
SET email = 'ana.santos@exemplo.pt'  
WHERE id_formando = 1;
```

Neste caso, apenas o formando com `id_formando` igual a 1 deve ser alterado.

8.6 Apagar dados com DELETE

O comando DELETE permite apagar registos. Tal como no UPDATE, a cláusula WHERE é essencial para indicar exatamente que registos devem ser apagados.

Exemplo:

```
DELETE FROM Formandos  
WHERE id_formando = 1;
```

Sem WHERE, o comando DELETE pode apagar todos os registos da tabela. Por isso, deve ser usado apenas depois de confirmar muito bem a condição.

Alerta importante: Antes de usar UPDATE ou DELETE, é recomendável fazer primeiro um SELECT com a mesma condição WHERE para confirmar que os registos selecionados são os corretos.

8.7 Comandos essenciais

Comando	Função	Exemplo de uso
SELECT	Consultar dados.	Ver formandos registados.
INSERT	Inserir novos registos.	Adicionar um formando.
UPDATE	Alterar dados existentes.	Corrigir um email.
DELETE	Apagar registos.	Remover um registo de teste.

8.8 Conceitos essenciais deste tema

Conceito	Explicação simples
T-SQL	Extensão do SQL usada no Microsoft SQL Server.
INSERT	Comando usado para inserir novos registos.
UPDATE	Comando usado para alterar registos existentes.
DELETE	Comando usado para apagar registos.

8.9 Atividade pratica

A atividade seguinte ajuda a praticar comandos básicos de manipulação de dados.

76. Considera uma tabela Produtos com id_produto, nome, preço e stock.
77. Escreve um INSERT para adicionar um produto.
78. Escreve um SELECT para confirmar que o produto foi inserido.
79. Escreve um UPDATE para alterar o preço do produto.
80. Escreve um DELETE para apagar apenas esse produto de teste.

Resultado esperado: No final, o formando deve conseguir distinguir comandos de consulta e comandos que alteram dados, usando WHERE com cuidado.

8.10 Verificação da aprendizagem

Responde por palavras tuas:

81. O que é Transact-SQL?
82. Para que serve o comando INSERT?
83. Porque o UPDATE deve normalmente usar WHERE?
84. O que pode acontecer se executar DELETE sem WHERE?
85. Porque é útil testar primeiro com SELECT?

Síntese do 8

Transact-SQL permite consultar e manipular dados no SQL Server. SELECT consulta dados, INSERT insere registos, UPDATE altera informação e DELETE apaga registos. Os comandos que alteram dados devem ser usados com cuidado, sobretudo com uma condição WHERE bem definida.

9. Filtragem e ordenação de dados

Quando uma tabela tem muitos registos, raramente queremos ver tudo ao mesmo tempo. Muitas vezes precisamos de consultar apenas os dados que respondem a uma pergunta concreta. Para isso, usamos filtros e ordenação.

A filtragem permite escolher que registos aparecem no resultado. A ordenação permite definir a ordem pela qual esses resultados são apresentados.

Ideia-chave: Uma boa consulta não mostra todos os dados: mostra os dados certos, na ordem certa.

9.1 Filtrar dados com WHERE

A clausula WHERE permite indicar uma condição. Apenas os registos que respeitam essa condição aparecem no resultado.

Por exemplo, se quisermos ver apenas os formandos maiores de idade, podemos usar uma condição sobre o campo idade.

- WHERE limita os registos apresentados.
- A condição deve comparar campos com valores.
- Podem ser usados operadores como =, >, <, >= e <=.
- Filtros ajudam a encontrar informação mais rapidamente.
- A mesma lógica de WHERE também aparece em UPDATE e DELETE, exigindo cuidado.

9.2 Operadores de comparação

Os operadores de comparação permitem construir condições. São usados para comparar o valor de um campo com outro valor.

Exemplos:

```
SELECT nome, idade
FROM Formandos
WHERE idade >= 18;
```

```
SELECT nome, localidade
FROM Formandos
WHERE localidade = 'Lisboa';
```

No primeiro exemplo, aparecem formandos com idade igual ou superior a 18. No segundo, aparecem apenas formandos cuja localidade seja Lisboa.

Nota: Valores de texto costumam ser escritos entre aspas simples. Valores numéricos não precisam de aspas.

9.3 Combinar condições com AND e OR

Podemos combinar condições para tornar a consulta mais precisa. O operador AND exige que as duas condições sejam verdadeiras. O operador OR permite que pelo menos uma das condições seja verdadeira.

```
SELECT nome, idade, localidade
FROM Formandos
WHERE idade >= 18 AND localidade = 'Lisboa';
```

```
SELECT nome, localidade
FROM Formandos
WHERE localidade = 'Lisboa' OR localidade = 'Porto';
```

No primeiro caso, o formando tem de cumprir as duas condições. No segundo, basta pertencer a uma das localidades indicadas.

- AND torna o filtro mais restritivo.
- OR torna o filtro mais abrangente.
- Usar parênteses quando houver várias condições.
- Ler a condição em linguagem natural antes de executar.
- Confirmar se a lógica corresponde ao resultado pretendido.

9.4 Pesquisar texto com LIKE

O operador LIKE permite procurar padrões em campos de texto. É útil quando não sabemos o texto completo ou quando queremos encontrar valores que começa, acabam ou contêm determinada sequência.

```
SELECT nome
FROM Formandos
WHERE nome LIKE 'Ana%';

SELECT nome
FROM Formandos
WHERE nome LIKE '%Silva%';
```

O símbolo % representa qualquer conjunto de caracteres. Assim, 'Ana%' procura nomes que comecem por Ana, enquanto '%Silva%' procura nomes que contenham Silva.

- LIKE é usado sobretudo com texto.
- % representa zero ou mais caracteres.
- O padrão 'A%' procura valores que começa por A.
- O padrão '%A' procura valores que terminam em A.
- O padrão '%A%' procura valores que contêm A.

Alerta: Filtros com LIKE podem devolver muitos resultados se forem demasiado abertos.

9.5 IN e BETWEEN

IN permite comparar um campo com uma lista de valores. BETWEEN permite filtrar valores dentro de um intervalo.

Exemplos:

```
SELECT nome, localidade
FROM Formandos
WHERE localidade IN ('Lisboa', 'Porto', 'Braga');

SELECT nome, idade
FROM Formandos
WHERE idade BETWEEN 18 AND 30;
```

IN é útil quando temos vários valores possíveis. BETWEEN é útil para intervalos, como idades, datas ou valores numéricos.

9.6 Ordenar dados com ORDER BY

ORDER BY permite definir a ordem dos resultados. Podemos ordenar de forma crescente com ASC ou decrescente com DESC.

Exemplo:

```
SELECT nome, idade, localidade
FROM Formandos
WHERE idade >= 18
ORDER BY nome ASC;
```

Neste exemplo, os formandos maiores de idade aparecem ordenados pelo nome, de A a Z.

Dica: Quando ASC não é indicado, muitos sistemas assumem ordenação crescente por defeito. Mesmo assim, escrever ASC ou DESC torna a consulta mais clara.

9.7 Resumo dos filtros e ordenação

Elemento	Função	Exemplo
WHERE	Filtra os registos.	WHERE idade >= 18
AND	Exige duas condições verdadeiras.	idade >= 18 AND ativo = 1
OR	Aceita uma de várias condições.	localidade = 'Lisboa' OR localidade = 'Porto'
ORDER BY	Ordena os resultados.	ORDER BY nome ASC

9.8 Conceitos essenciais deste tema

Conceito	Explicação simples
WHERE	Clausula usada para filtrar registos.
AND	Operador que exige que duas condições sejam verdadeiras.
OR	Operador que aceita uma de várias condições.
ORDER BY	Clausula usada para ordenar resultados.

9.9 Atividade pratica

A atividade seguinte ajuda a construir consultas com filtros e ordenação.

86. Considera uma tabela Produtos com nome, categoria, preço e stock.
87. Escreve uma consulta para mostrar produtos com stock superior a 0.
88. Filtra produtos da categoria 'Informática'.
89. Mostra produtos com preço entre 10 e 50.
90. Ordena os resultados pelo preço, do mais barato para o mais caro.

Resultado esperado: No final, o formando deve conseguir usar WHERE, operadores lógicos e ORDER BY para obter resultados mais precisos.

9.10 Verificação da aprendizagem

Responde por palavras tuas:

91. Para que serve a clausula WHERE?
92. Qual é a diferença entre AND e OR?
93. Para que serve o operador LIKE?
94. Quando podes usar BETWEEN?
95. Qual é a função do ORDER BY?

Síntese do 9

A filtragem e a ordenação tornam as consultas mais uteis. WHERE permite seleccionar apenas os registos pretendidos, operadores como AND, OR, LIKE, IN e BETWEEN refinam as condições, e ORDER BY organiza os resultados para facilitar a leitura e a análise.

Conclusão

Ao longo da UFCD 10788 - Fundamentos da linguagem SQL, foram trabalhados os conceitos essenciais para compreender, planear e utilizar bases de dados relacionais. O percurso começou pela noção de base de dados e terminou na construção de consultas com filtragem e ordenação.

O objetivo principal foi criar uma base solida para que o formando consiga ler, interpretar e escrever comandos SQL simples, percebendo sempre a relação entre os dados guardados e as instruções executadas.

Ideia-chave: Aprender SQL e aprender a fazer boas perguntas aos dados.

1. O que foi trabalhado

O manual foi organizado por partes para permitir uma aprendizagem gradual. Primeiro foram apresentados os conceitos gerais; depois, a estrutura das bases de dados relacionais; por fim, os comandos SQL essenciais.

- Compreensão do conceito de base de dados.
- Identificação de ambientes e ferramentas de bases de dados.
- Utilizes da terminologia relacional.
- Planeamento de tabelas, campos, chaves e relações.
- Primeiras instruções SQL para criar, consultar e manipular dados.

2. Competências desenvolvidas

No final desta UFCD, o formando devera sentir-se mais preparado para compreender bases de dados relacionais e executar tarefas simples com SQL.

Área trabalhada	O que ficou consolidado	Aplicação pratica
Bases de dados	Organização de dados em estruturas coerentes.	Identificar dados e informação.
Modelo relacional	Tabelas, campos, registos, chaves e relações.	Desenhar pequenas bases de dados.
SQL/T-SQL	Criar, consultar e manipular dados.	Executar comandos simples.
Consultas	Filtrar e ordenar resultados.	Responder a perguntas concretas.

Nota: O objetivo não é decorar todos os comandos, mas compreender a lógica e saber consultar exemplos, testar e corrigir com método.

3. Cuidados importantes ao trabalhar com SQL

SQL permite fazer muito mais do que consultar dados. Alguns comandos alteram informação, criam estruturas ou apagam registos. Por isso, é importante trabalhar com atenção e confirmar sempre o impacto de cada instrução.

- Usar bases de dados de treino para praticar.
- Ler o comando antes de executar.
- Testar filtros com SELECT antes de usar UPDATE ou DELETE.
- Guardar scripts importantes com nomes claros.
- Pedir apoio quando uma instrução puder alterar muitos dados.

Alerta: Comandos como UPDATE e DELETE devem ser usados com especial cuidado, principalmente quando não existe uma condição WHERE clara.

4. Exemplo de percurso pratico final

Como consolidação, o formando pode realizar uma atividade final simples: criar uma pequena base de dados de treino, criar tabelas, inserir dados e executar consultas.

```
CREATE DATABASE BD_TreinoSQL;
GO

USE BD_TreinoSQL;
GO

CREATE TABLE Formandos (
    id_formando INT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    idade INT,
    localidade VARCHAR(80)
);
```

96. Criar a base de dados.
97. Criar uma tabela.
98. Inserir dados de teste.
99. Consultar todos os dados.
100. Aplicar filtros e ordenação.

5. Autoavaliação

A autoavaliação ajuda o formando a perceber o que já domina e o que deve continuar a praticar. As perguntas seguintes podem ser usadas no final da UFCD.

101. Consigo explicar o que é uma base de dados relacional?
102. Consigo distinguir tabela, campo, registo, chave primaria e chave estrangeira?
103. Consigo criar uma base de dados e uma tabela simples?
104. Consigo escrever uma consulta SELECT com filtros?
105. Consigo identificar comandos que alteram dados e usar cuidados antes de os executar?

6. Continuidade da aprendizagem

Depois desta UFCD, a aprendizagem pode continuar com temas como consultas com várias tabelas, joins, funções de agregação, subconsultas, views, procedimentos armazenados e administração básica de bases de dados.

- Rever os exemplos do manual.
- Reescrever consultas sem olhar para a solução.
- Criar pequenas bases de dados de treino.

- Explicar os comandos em linguagem natural.
- Usar erros como oportunidade para compreender melhor a sintaxe.

Mensagem final

A linguagem SQL é uma ferramenta essencial para trabalhar com informação. Mais do que escrever comandos, importa compreender o que os dados representam e que pergunta queremos responder. Com prática, rigor e curiosidade, SQL torna-se uma forma clara e poderosa de transformar dados em informação útil.

Bibliografia

As referências seguintes podem ser usadas para enquadramento, consulta e aprofundamento dos temas trabalhados nesta UFCD.

- Catálogo Nacional de Qualificações. UFCD 10788 - Fundamentos da linguagem SQL.
- Microsoft Learn. Documentação de Transact-SQL e SQL Server.
- Oracle. SQL Language Reference.
- PostgreSQL Global Development Group. PostgreSQL Documentation - SQL.
- W3Schools. SQL Tutorial, para consulta introdutória e exemplos simples.
- Silberschatz, A., Korth, H. F., e Sudarshan, S. Database System Concepts.

Referencia de enquadramento: Catálogo Nacional de Qualificações, UFCD 10788 - Fundamentos da linguagem SQL.